

Communication protocols of IoT

Protocols	MQTT	CoAP	XMPP	SOAP	UPnP
XML based	No	No	Communication protocol for message-oriented middleware based on XML	Relies on XML information set for its message format	No
TCP/UDP	TCP	UDP	TCP	Both TCP and UDP	UDP
IPv4/IPv6	IPv4/IPv6	IPv4/IPv6	IPv4/IPv6	IPv4	IPv4/IPv6
M2M support	OASIS standard	ETSI standard	—	—	—
USAGE	From pervasive devices to servers	Simple electronic devices, Resource-constrained devices	Video transfer	Implementation of Web services	Permits networked devices to seamlessly discover others

Sensors are at the lowest level in an IoT system. These can be temperature sensors, position sensors, velocity sensors, etc. They are connected to the nearest point to track data.

Once the data is collected from the objects through the sensors, it is processed and used in various applications. Application areas include surveillance, clinical infrastructure monitoring, health monitoring, smart transportation, etc.

IoT protocol layers

Now let us look at IoT's working in a bit more detail. An IoT system comprises an application layer, information processing, a network, and sensing and identification. All these four components together make an IoT solution.

The hardware for an IoT system has the following.

Input/output devices: Sensors, actuators, LEDs, relays, motors, linear actuators, lasers, solenoids, speakers, LCD, plasma displays, robots, etc.

Network devices: Modem, gateway, router, satellite, tower, etc.

Processing devices: Grid, cloud, embedded processor, quantum computing, etc.

Memory devices: Cloud memory, flash memory and quantum memory.

The software for an IoT system requires a wireless sensor network (WSN) that has following characteristics:

- Smart and autonomous
- Can be configured automatically
- Self-monitoring and self-healing

- Capable of anomaly detection and tracking

The event notification service (ENS) of an IoT system acts as a common collector and distributor of events. Events come from multiple heterogeneous sources. Devices can subscribe to specific subsets of events.

In publish-subscribe software architectural style of ENS, whenever there is a change in an event, a notification is published stating the same. Then, accordingly, messages are generated to be sent to the user. For example, for a voltage change or current change, the event will be interpreted as switching on or off. This will be then conveyed to the subscriber (user) to act upon.

In object as a (Web) service ENS, this is an emerging paradigm. Nowadays, services such as food delivery happen over the Internet. Moving forward, thanks to AI, robots will be aligning themselves with Web technology to provide this service in a faster and efficient manner.

UPnP+ protocol for IoT

This protocol is extensively used in present-day high-end IoT solutions. Going forward, it will be required in cognitive IoT because the objects adhering to this protocol can interact better.

UPnP+protocol involves following steps:

By: Dr Manjunath Iyer

The author has over two decades of experience and has filed about 82 patents, authored 180 research papers and a book.

The article was originally published in the June 2020 issue of Electronics For You.

Discovery: Device advertises itself within a network and enables discovery.

Description: After the discovery, the control point hits the URLs to get the description.

Addressing: Then the address is auto-generated for ease of use.

Presentation: The focus is on human-machine interfaces.

Eventing: This step is closely coupled with control actions.

Control: Here the device is required to perform some action over other devices.

However, some issues are faced due to the presence of a multitude of IoT devices. In a large distributed system this can sometimes create confusion. Besides, sensors that are placed on a moving object are not made robust enough and at times provide low output.

Issues can arise because of:

- Multiple services, multiple users
- Heterogeneous services
- Bursty traffic requests, that is, people sending large amounts of data to the cloud at the same time. This causes an overload resulting in system/server crash.

How to handle these issues?

Do not put all applications to use at once. Put some applications to use first so that the moment the device captures data, it also processes it. And divide the data into parts before transferring it to the cloud. 